

Phase 11 Checklist Status

Date: May 15, 2026

Source of truth: `run_phase11_status.py --recent-hours 24 --json`, `run_runtime.py --check-only`, `run_phase11_runtime_proof.py --json`, `run_runtime_status.py --recent-hours 24`, and `run_runtime_storage_status.py --json`.

Purpose: Convert the Phase 11 runtime-revision plan into a direct implementation checklist with explicit status labels.

Status Legend

- **Implemented:** the repo has the required code and documentation behavior, and the requirement is satisfied at the level the Phase 11 document asks for.
- **Partially implemented:** the repo has the structural code or documentation changes, but the operational proof is incomplete.
- **Not implemented:** the requirement is still blocked by missing operational evidence or unresolved machine state.

Checklist Snapshot

Status	Requirement	Current truth
Implemented	Canonical runtime activation path exists	<code>run_runtime.py</code> is now the documented launcher and <code>polymarket.plist</code> points to it.
Implemented	Background runtime path is environment-safe	Runtime env loading now supports <code>.env.runtime</code> and repo-ignored <code>.env.runtime.secrets</code> .
Implemented	Durable Phase 3 operation is restored for bounded proof	Durable state pre-flight succeeds with <code>backend=sqlite</code> ; the current inspected DB shows <code>checkpoint_count_recent=2</code> and <code>candidate_count_recent=1</code> .
Implemented	Phase 4 live story is explicit	Phase 4 is documented as the canonical downstream live path for <code>alert.live</code> .

Implemented		Phase 6 live story is explicit	Phase 6 is documented as an optional shadow-only extension rather than the default runtime; the inspected DB now has <code>shadow_score_count_recent=1</code> .
Implemented		Archived-window replay path exists	<code>run_runtime_replay_window.py</code> plus the Phase 11 storage runbook provide the explicit replay path.
Implemented		Storage lifecycle tooling exists	Retention, restore, compaction planning, and disk-pressure guards exist; after SSD backup and local archive pruning, free headroom is about 132G.
Partially implemented	imple-	Plaintext-secret dependence is removed from the canonical workflow	The workflow now supports safe secret paths, but real credential rotation is still an external operational task.
Implemented		Truthfulness refresh exists	Current-state and archive-loss memos now state the real runtime and replay truth explicitly.
Implemented		Durable Phase 3 state backend preflight is healthy	<code>run_runtime.py</code> <code>--env-file</code> <code>.env.runtime.example</code> <code>--check-only</code> reports a successful Phase 3 state-backend preflight with <code>backend=sqlite</code> .
Implemented		DB shows fresh recent Phase 3 candidates and checkpoints	The current inspected 24-hour window shows <code>candidate_count_recent=1</code> and <code>checkpoint_count_recent=2</code> .
Implemented		DB shows fresh recent Phase 4 evidence, alerts, and deliveries	The current inspected 24-hour window shows <code>evidence_query_count_recent=2</code> , <code>alert_count_recent=1</code> , and <code>delivery_attempt_count_recent=1</code> .

Implemented	DB shows fresh recent Phase 6 shadow scoring	The current inspected 24-hour window shows <code>shadow_score_count_recent=1</code> after pausing the background collector and running one bounded shadow-scoring pass.
Implemented	Archived windows can be replayed intentionally	The operator no longer has to rely on the live hour-only tailer for historical windows.
Implemented	Machine can run bounded proof work under the local proof profile	The storage guard now reports warning only because the retained SQLite DB keeps managed storage slightly above the warning threshold; local free headroom is no longer the blocker.
Implemented	Final docs distinguish live proof from historical or replay-only evidence	The runtime-status, current-state memo, and archive-loss memo now say this plainly.

Remaining Required Actions

The main remaining actions are operational rather than architectural:

- completed: back up the local data-bearing machine's Phase 11 DB, reports, docs, raw data, and detector-input archives to the attached SSD,
- completed: prune the local `data/raw` and `data/detector_input` archive copies after backup, then recreate empty runtime directories,
- optionally run a longer organic live-market window after storage headroom improves and the collector is restarted,
- keep the May 15 proof wording honest: it used `synthetic_detector_input_fallback` because the expected April seed partitions were missing locally,
- rotate any previously exposed real credentials outside the repository.

Data Ownership Constraint

The current project state is asymmetric across collaborators:

- The primary operator machine has the large local runtime state: roughly 100G under `data/` and 27G under `database/`.
- A collaborator without that data can still work on documentation, code review, presentation packaging, runbook cleanup, CLI dry runs against examples, and small seeded proof paths.

- The collaborator cannot honestly prove fresh live Phase 3 or Phase 4 runtime health unless they receive a database/archive subset or the proof is run on the data-bearing machine.
- Therefore, final Phase 11 operational proof must be produced on the data-bearing machine, while cleanup, review, and packaging can be parallelized on a code-only clone.

Four-Day Closeout Plan

Day	Primary operator with data	Friend without full data
1	Freeze current evidence, back up or preserve the SQLite DB, inspect storage status, and free enough disk to pass the Phase 11 storage guard.	Review Phase 11 docs for truthfulness, remove stale Redis-only wording, prepare final report/presentation structure, and verify commands against <code>.env.runtime.example</code> .
2	Completed: <code>run_runtime.py --check-only</code> passed under the local proof profile, and <code>run_phase11_runtime_proof.py --json</code> produced fresh Phase 3 and Phase 4 rows.	Review collector, detector, and Phase 4 paths for obvious blockers; prepare screenshots/tables explaining configured vs observed runtime.
3	Completed: copy the SQLite DB, report artifacts, Phase 11 docs, raw archives, and detector-input archives to the attached SSD, then prune local archive copies and rerun storage checks. Next, package proof artifacts and only then optionally run a longer organic live-market window.	Package proof artifacts, update final checklist language, and prepare the friend handoff explaining which artifacts require the original data machine.
4	Re-run <code>run_runtime_status.py</code> , <code>run_runtime_storage_status.py</code> , and <code>run_phase11_status.py</code> ; commit final docs and evidence artifacts.	Final read-through, presentation polish, and consistency check against SRS/Phase 8–10 claims.

Final Stretch Goal

The realistic final stretch goal is not to prove a profitable autonomous trading system. The final stretch goal is to deliver a truthful, reproducible local-first research/runtime project with:

- a working canonical runtime launcher,
- safe environment and secret handling,
- disk-pressure protection,
- replay-aware storage documentation,
- fresh Phase 3 bounded runtime evidence,
- Phase 4 evidence/alert proof from bounded proof input,
- one Phase 6 shadow-score proof row,
- clear separation between live runtime proof, replay proof, seeded proof, and offline reports.

Can This Be Fixed In Four Days?

Yes, if the target is Phase 11 closeout and project submission quality. The remaining work is feasible because the hard architectural pieces already exist. The critical path is operational: disk headroom, one successful canonical run, generated status artifacts, and truthful final documentation.

No, if the target is a new production-grade always-on trading or ML system. That would exceed Phase 11 scope and should not be claimed.

Bottom Line

Phase 11 is **implemented for project closeout with bounded proof**. The repo now has the right startup path, replay path, storage path, secret posture, truthfulness posture, a fresh bounded Phase 3-to-Phase 4 proof, one current Phase 6 shadow-score row, an attached-SSD backup of the DB, report artifacts, raw archives, and detector-input archives, and restored local disk headroom after pruning the backed-up archive copies. What it still lacks is a long organic live-market run under comfortable storage conditions. The next step is to package the final evidence and avoid overclaiming beyond the bounded proof.